

AMBIENTE DE DESENVOLVIMENTO INTEGRADO AURORA: COMPILAÇÃO E SIMULAÇÃO PARA O PROCESSADOR SOFT-CORE SAPHO

Chrysthofer A. A. Afonso[1] - chrysthofer.afonso@estudante.ufjf.br
Gabriel A. Torres[1] - gabriel.aquino@estudante.ufjf.br
Pedro H. S. R. Santos[1] - pedro.romeu@estudante.ufjf.br
Arthur A. Martins[1] - arthur.araujo@estudante.ufjf.br
Nelson C. N. de Assis[1] - nelson.assis@estudante.ufjf.br
Guilherme B. Morett[2] - guilherme.morett@iprj.uerj.br
Luciano M. de Andrade Filho[1] - luciano.andrade@ufjf.br
[1]Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN),
Universidade Federal de Juiz de Fora – Juiz de Fora – MG
[2]Instituto Politécnico, Universidade do Estado do Rio de Janeiro – Nova Friburgo – RJ

Resumo. *O projeto de sistemas embarcados em FPGA é, em essência, um ciclo de modelagem computacional: descreve-se o algoritmo, sintetiza-se o modelo estrutural e simula-se o resultado antes de comprometê-lo em hardware. O processador dinâmico SAPHO, que sintetiza apenas os recursos exigidos por cada aplicação, dependia de uma interface (IDE V1) que encadeava seis ferramentas à mão. Este trabalho apresenta a AURORA, ambiente de desenvolvimento integrado e multiplataforma, construído sobre Electron e distribuído em código aberto, que unifica o ciclo numa única operação: compilação, simulação com inspeção das formas de onda (Icarus e GTKWave) e visualização do RTL em blocos (Yosys e PRISM), sem software proprietário, requerido apenas na implementação física. A avaliação mostra que a automação reduz o esforço de operação de seis comandos manuais a um, enquanto o custo de máquina da cadeia, abaixo de 0,7 s, é dominado pela etapa de elaboração do modelo, e não pela simulação. Em simulações longas, avaliadas até um sistema de dois processadores, é o tempo simulado que passa a governar esse custo.*

Keywords: *Modelagem e simulação de sistemas digitais, FPGA, Processador soft-core, Automação de fluxo de projeto, AURORA IDE.*

1. INTRODUÇÃO

As FPGAs (*Field-Programmable Gate Arrays*) consolidaram-se como plataformas relevantes para o processamento de cálculos complexos, destacando-se pelo paralelismo massivo e pela eficiência energética (Compton and Hauck, 2002). Em cenários de alta exigência, como no experimento ATLAS do CERN (Organização Europeia para a Pesquisa Nuclear, em francês *Conseil Européen pour la Recherche Nucléaire*), esses dispositivos constituem o núcleo dos sistemas de gatilho (*trigger*) e de aquisição de dados, que filtram as colisões de partículas em tempo real (ATLAS Collaboration, 2008).

Quando o projeto demanda algoritmos extensos ou cálculos em ponto flutuante, recorre-se a processadores embarcados (*soft-cores*), cuja arquitetura padrão, instanciada por inteiro independentemente do uso, desperdiça área lógica da FPGA. Para contornar isso, desenvolveu-se o SAPHO (*Scalable-Architecture Processor for Hardware Optimization*), cuja arquitetura é gerada sob demanda (Kapisch et al., 2026): programado em C± (lê-se "C mais-menos"), uma linguagem de médio nível inspirada em C e estendida às necessidades do NIPS-CERN, ele sintetiza apenas os recursos de *hardware* exigidos pela aplicação.

Esse fluxo constitui, na prática, um ciclo iterativo de modelagem: o algoritmo em C± é o *modelo funcional*; a cadeia de compilação o converte em um *modelo estrutural* em nível de transferência de registradores (RTL, *Register-Transfer Level*); e a simulação confronta esse modelo com o comportamento esperado, antes de comprometê-lo em *hardware*. A agilidade com que se percorre esse ciclo determina o ritmo da investigação, como no simulador de pulsos do TileCal em tempo real construído sobre o SAPHO em FPGA (Ribeiro, 2025; Ribeiro and Andrade Filho, 2025).

Esse ciclo, porém, dependia da interface predecessora, a Versão 1 (V1): um protótipo monolítico (todo o programa reunido num único bloco, sem módulos independentes) em C# (lê-se "C-Sharp"), restrito ao Windows, que gerava a descrição em *Assembly* e Verilog (Kapisch et al., 2026), mas não integrava verificação, simulação nem visualização. Simular exigia operar à mão uma sucessão de ferramentas de linha de comando, um fluxo lento, propenso a erros e de difícil reprodução.

A AURORA (*Advanced Utility Running Optimized Resource Architectures*), um ambiente de desenvolvimento integrado (IDE, do inglês *Integrated Development Environment*) construído sobre a tecnologia *Electron* e distribuído em código aberto, elimina esse atrito: pela função *Full Build*, um único comando percorre o ciclo inteiro, da compilação em C± à simulação com inspeção das formas de onda e à pré-visualização do circuito RTL, orquestrando ferramentas livres e restringindo o *software* proprietário à etapa final de implementação física (Seção 5.3). Desenvolvida pelo Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN)[1], da UFJF, a AURORA já sustenta pesquisas de pós-graduação e foi adotada na cadeira de Dispositivos Lógicos Programáveis (DLP) da graduação em Engenharia Elétrica.

As principais contribuições são: (i) um ambiente de modelagem aberto que unifica, em um único comando, a compilação, a simulação funcional (Icarus e GTKWave) e a visualização do RTL em blocos (Yosys e PRISM), sem recorrer a software proprietário até a implementação física, o que torna o ciclo completo acessível ao ensino e à pesquisa sem licenças; (ii) a automação que reduz a operação de seis passos manuais na V1 a um; e (iii) uma avaliação quantitativa que separa o *esforço de operação* do usuário do *custo de máquina* da cadeia e mostra que este é dominado pela elaboração do modelo, não pela simulação, estendida a um sistema de dois processadores.

2. TRABALHOS RELACIONADOS

As ferramentas que sustentam o ciclo de modelagem em FPGA repartem-se entre ambientes proprietários integrados e utilitários abertos especializados. Os ambientes dos fabricantes, como o Quartus Prime (Intel Corporation, 2025) e o Vivado (AMD, 2025), cobrem da síntese ao roteamento físico, porém são de código fechado, caros e atrelados aos dispositivos de cada fabricante; ademais, foram concebidos para o fluxo de RTL convencional, não para uma cadeia

[1] Em parceria com o CERN. Mais informações em `https://www.nipscern.com/projects/aurora`.

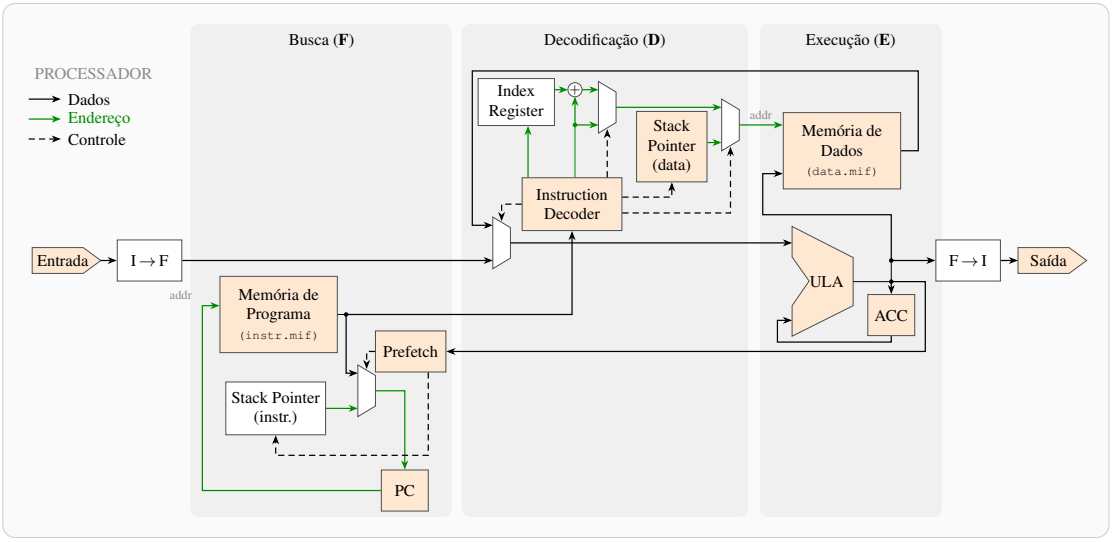


Figura 1: Datapath do SAPHO com o *pipeline* de três estágios, busca (F), decodificação (D) e execução (E): a ULA combinacional acumula no ACC e a condição de desvio (|ACC|) realimenta o Prefetch, sem quebras na cadeia.

Fonte: Autores (2026)

própria (C± → *Assembly* → Verilog) que precede a síntese física.

No campo do código aberto, ferramentas maduras cobrem etapas isoladas do ciclo (Icarus Verilog (Williams, 2024) e Verilator (Snyder, 2024) na simulação, Yosys (Wolf, 2024) na extração do RTL e GTKWave (Bybell, 2024) nas formas de onda), mas operadas isoladamente pela linha de comando, com formatos próprios, cabendo ao usuário costurar as transições. Quanto aos núcleos embarcados, os comerciais (NIOS II) e os abertos (RISC-V) instanciam arquiteturas fixas; o SAPHO gera a sua sob demanda (Kapisch et al., 2026).

É nesse espaço que se posiciona a AURORA: aberta diante dos ambientes proprietários e integrada diante das ferramentas isoladas, uma combinação ausente nas alternativas existentes.

3. O PROCESSADOR SAPHO

O SAPHO é um processador do tipo *soft-core* desenvolvido pelo laboratório NIPS-CERN (Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN), 2026), no qual os recursos são alocados automaticamente em tempo de compilação. A consequência é relevante do ponto de vista de modelagem: o modelo funcional escrito pelo usuário não é apenas executado sobre uma máquina pré-existente, ele *determina* a máquina, pois somente os elementos lógicos exigidos pelo algoritmo são instanciados no *hardware* (Kapisch et al., 2026). A robustez desse modelo já foi validada em aplicações de alta carga, da geração de fractais em FPGA (Andrade Filho et al., 2025) à simulação de pulsos do TileCal em tempo real para o experimento ATLAS (Ribeiro, 2025; Ribeiro and Andrade Filho, 2025); frente ao núcleo comercial NIOS II, o SAPHO alcança menor custo de *hardware* e menor tempo de execução (Kapisch et al., 2026).

A arquitetura reúne uma unidade de controle, uma unidade lógica e aritmética (ULA) parametrizada pelo compilador a partir das definições do usuário (largura da palavra, número de portas) e capaz de executar cada operação em um único ciclo, e uma unidade de memória com endereçamento relativo, instanciada apenas quando o programa emprega vetores ou matrizes. Combina, assim, a definição do *hardware* por *software* (como em *High Level Synthesis*, HLS) com a execução de um processador embarcado.

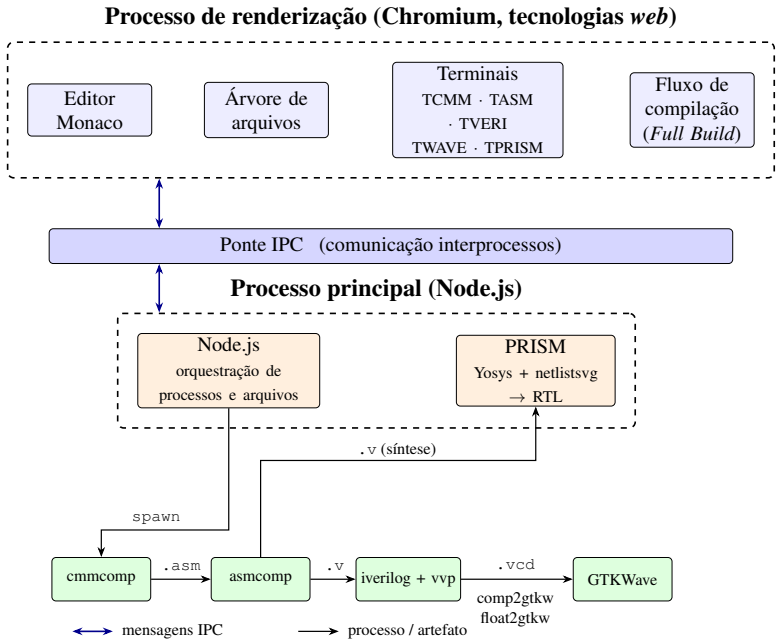


Figura 2: Arquitetura da AURORA sobre o *Electron*: o processo de renderização (Chromium) concentra a interface, e o processo principal (Node.js) orquestra, pela ponte IPC, as ferramentas do *Full Build*, com o PRISM extraindo a hierarquia via Yosys.
Fonte: Autores (2026)

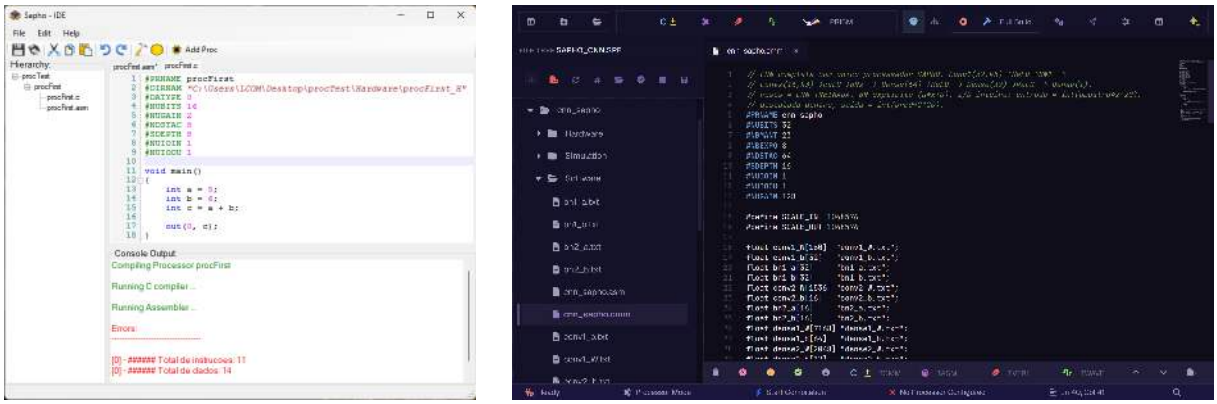
Para reduzir a latência, a arquitetura emprega um *pipeline* de três estágios síncronos, busca (F), decodificação (D) e execução (E), detalhados na Figura 1. Seu diferencial é executar qualquer algoritmo sem interromper a cadeia $F \rightarrow D \rightarrow E$, mesmo em saltos condicionais ou chamadas de função (Kapisch et al., 2026). A passagem do modelo funcional ao estrutural cabe à cadeia de compiladores YANC, detalhada na Seção 5.2; historicamente operada pela IDE V1, foram as suas limitações que motivaram a arquitetura discutida a seguir.

4. BASE TECNOLÓGICA DA IDE

Interfaces de ferramentas de *Electronic Design Automation* (EDA) dependiam, historicamente, de linguagens compiladas e de APIs proprietárias de cada sistema operacional, como o .NET/C++ da IDE V1, o que gerava aprisionamento tecnológico (*vendor lock-in*). Optou-se, por isso, pelo desenvolvimento *desktop* multiplataforma sobre tecnologias *web*; como os navegadores executam o JavaScript em ambiente isolado (*sandbox*), sem acesso ao sistema de arquivos, o *framework Electron* (Yang et al., 2025) contorna o impasse ao unir, num único executável, o motor de renderização Chromium (a interface) e o ambiente Node.js (o sistema operacional), ambos sobre o motor JavaScript V8 e conectados por comunicação interprocessos (IPC), conforme a Figura 2. É essa separação que permite à IDE invocar compiladores e simuladores como processos filhos.

5. A IDE AURORA

Sucessora integral da V1, a AURORA é hoje a única interface gráfica do ecossistema do NIPS-CERN: o *frontend* que orquestra os compiladores YANC (*Yet Another Compiler*) e entrega, ao final, a descrição de *hardware* do processador SAPHO, distinguindo-se das ferramentas EDA tradicionais, focadas na síntese e no roteamento físico. A Figura 3 contrasta a interface da predecessora V1 com a AURORA em operação. A AURORA organiza o trabalho



(a) (b)

Figura 3: Interfaces do ecossistema: (a) a IDE V1, monolítica, em C#, com um único console; (b) a AURORA em operação no Modo Processador, com a árvore do projeto, as diretivas de *hardware* do C± e os terminais dedicados.

Fonte: Autores (2026)

em duas frentes, que correspondem a dois níveis de modelagem: o Modo Processador, no qual se constrói o modelo de uma unidade de processamento isolada, e o Modo Projeto, no qual esses modelos são compostos em um sistema.

5.1 Modos de trabalho e linguagens suportadas

No Modo Processador, o usuário fixa os parâmetros que delimitam o espaço de representação do modelo, visíveis na Figura 3(b): número de *bits*, tamanho da mantissa e do expoente e quantidade de portas. O editor integrado suporta três linguagens: o C±, com suporte nativo à notação *bra-ket* da física de altas energias (Dirac, 1958); o *Assembly*, para otimizações de baixo nível; e o Verilog. No Modo Projeto, os processadores já modelados são instanciados e interconectados em um *pipeline* completo; é também aqui que se define o *testbench*, o cenário de estímulos que fecha a especificação para a validação conjunta.

5.2 *Toolchain* YANC e fluxo de compilação automatizado

O núcleo de tradução do ecossistema é a cadeia YANC, escrita em C com o auxílio das ferramentas Flex e Bison (Levine, 2009); é ela que leva o modelo funcional ao estrutural, dimensionando memórias, ULA e portas conforme o algoritmo. A AURORA abstrai essa cadeia por uma interface em que o usuário define os parâmetros de compilação (modo paralelo ou seriado, frequência de *clock* e número de ciclos) e aciona a rotina automaticamente. O encadeamento é: `cmmcomp` traduz o C± (`.cmm`) para *Assembly*; `appcomp` o pré-processa, levantando a demanda de memória e de circuitos; `asmcomp` gera o Verilog e o *testbench*; e o `iverilog` elabora o projeto.

5.3 Validação do modelo: simulação e visualização RTL

Obtido o modelo estrutural, resta confrontá-lo com o comportamento esperado: a AURORA invoca o motor `vvp` do Icarus Verilog e processa as saídas pelos conversores `comp2gtkw` e `float2gtkw`, que as consolidam num arquivo de despejo de variáveis (`.vcd`). O fluxo culmina na análise visual via GTKWave, que valida a integridade lógica e a cronologia dos sinais.

Enquanto a simulação expõe o comportamento do modelo no tempo, a inspeção estrutural revela a sua topologia. Para essa segunda via, a AURORA aciona o PRISM (*Processor Rendering Interface for Schematic Models*), visualizador RTL que compartilha a mesma base

tecnológica: ele invoca o Yosys (*Yosys Open SYnthesis Suite*) para extrair a hierarquia do circuito, convertida pela ferramenta `netlistsvg` em diagramas vetorizados, como o da Figura 4. A representação destina-se à *leitura* do modelo, o RTL que o algoritmo gerou visto em blocos, e não à implementação física.

Com as duas vias acionadas de dentro da IDE, o projetista percorre todo o ciclo de *modelagem e verificação* sem sair do ambiente aberto; o *software* proprietário do fabricante só volta a ser requerido na *implementação física* (síntese, *fitting* e gravação).

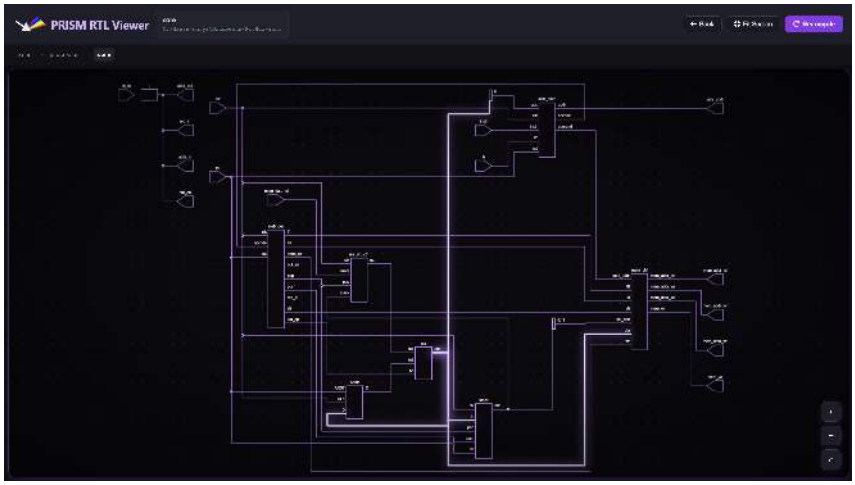


Figura 4: Visualização RTL no PRISM: o *netlist* do núcleo de um processador SAPHO, extraído pelo Yosys e desenhado em blocos e conexões, deixa ler diretamente a estrutura que o algoritmo em C± gerou, dos registradores de busca à ULA e à memória de dados.
Fonte: Autores (2026)

6. RESULTADOS E DISCUSSÃO

Esta seção avalia quantitativamente o principal ganho da AURORA frente à V1: a automação do fluxo de projeto. O experimento contrasta os dois percursos para levar um mesmo projeto de C± até a inspeção das formas de onda, a saber, a cadeia manual da V1 e a função *Full Build* da AURORA, em três projetos de porte crescente em código-fonte, todos disponíveis no conjunto de exemplos do SAPHO: o cálculo do seno (46 linhas de C±), um filtro adaptativo RLS (*Recursive Least Squares*, 65 linhas), técnica que reestima os coeficientes do filtro a cada nova amostra (Haykin, 2002), e uma FFT (122 linhas).

Cabe antes delimitar o que se pode atribuir à IDE, distinguindo duas grandezas. O *esforço de operação* mede o trabalho do *usuário*: o número de vezes que ele precisa acionar uma ferramenta à mão para levar o projeto do código-fonte à forma de onda; independe da máquina. O *custo de máquina* mede o trabalho do *computador*: o tempo de processamento que a máquina gasta executando as ferramentas da cadeia (compilar, pré-processar, montar, elaborar e simular), independentemente de quem as aciona. Todos os tempos são de parede (*wall clock*), o tempo real decorrido em cada etapa. As medições foram feitas em um Intel Core Ultra 7 155H (16 núcleos, 22 *threads*), 15,5 GB de RAM, Windows 11, com Icarus Verilog 13.0 e a cadeia YANC 5.2, sob a mesma versão da AURORA (v4.1.12) para os três exemplos, executando-se cada configuração sete vezes e descartando a primeira; reporta-se a mediana das seis restantes.

6.1 Automação do fluxo de projeto

Na V1, essa cadeia de seis ferramentas era acionada à mão, localizando arquivos intermediários e reintroduzindo parâmetros a cada transição; a função *Full Build* da AURORA

a reduz a um único comando. A cadeia atual acrescentou o pré-processador `appcomp`, inexistente na V1, contabilizado no custo de máquina mas que nunca foi passo manual.

A Tabela 1 reúne as duas grandezas para esses três programas: no bloco superior, o *esforço de operação*; no inferior, o *custo de máquina* por etapa. Como as ferramentas são as mesmas nas duas rotas, o custo de máquina é comum a ambas; o que a AURORA altera é apenas o número de acionamentos manuais, de seis para um.

Tabela 1: Esforço de operação (invocações manuais) e custo de máquina por etapa (ms de parede), para três programas de exemplo de porte crescente. O custo de máquina é comum às rotas V1 e AURORA, pois executam as mesmas ferramentas; a V1 as aciona à mão, a AURORA em um único comando. Medianas de seis execuções.

	Programa de exemplo		
	Seno (46 lin.)	RLS (65 lin.)	FFT (122 lin.)
Esforço de operação (número de invocações manuais)			
IDE V1 (cadeia manual)	6	6	6
AURORA (*Full Build*)	1	1	1
Custo de máquina por etapa (ms, comum a V1 e AURORA)			
C± (`cmmcomp`)	54	54	49
Pré-processamento (`appcomp`)	50	51	46
Assembly→Verilog (`asmcomp`)	62	57	53
Elaboração (`iverilog`)	242	267	270
Simulação (`vvp`)	200	211	137
Total	608	640	555

Fonte: Autores (2026)

Da Tabela 1 decorrem três observações. Primeira, o ganho de operação é integral e independe do projeto: as seis invocações manuais da V1 reduzem-se a uma, suprimindo as buscas por arquivos e reintroduções de parâmetros entre elas. Segunda, as cinco etapas medidas custam menos de 0,7 s nos três exemplos, piso baixo o bastante para que o tempo de máquina não seja o fator limitante do ciclo de projeto, o que reforça que o valor da automação está em suprimir passos manuais, não em economizar processamento.

Terceira, o custo não é dominado pela simulação, como se poderia supor, e sim pela elaboração do projeto pelo `iverilog` (242 a 270 ms, de 40% a 49% do total). O custo tampouco acompanha o tamanho do programa: a FFT, o maior dos três fontes, apresenta o menor tempo total. A elaboração processa a biblioteca HDL do SAPHO, comum a todos os projetos, enquanto o programa do usuário responde por fração pequena do trabalho; o custo dependente da aplicação só emerge em simulações mais longas, quantificadas a seguir.

6.2 Custo da simulação em cargas crescentes

O custo da Tabela 1 isola o `vvp`, sem as formas de onda; resta o custo da simulação completa, tal como o *Full Build* a executa, com o despejo de todos os sinais. Mediu-se o tempo de parede em três cenários, em uma segunda máquina, a estação do laboratório (Dell XPS 8930, Windows) com a AURORA v4.1.12. Os cenários usam as cargas completas de dados: mil pontos no `Seno` (880,2 μs simulados) e vinte iterações da notação de Dirac no `proc_rls` (61,2 μs), e o `top_level` da Seção 6.3 reúne dois processadores por 7,0 ms simulados.

A proporcionalidade entre custo e tempo simulado é nítida: as medianas de 0,87 s, 10,09 s e 87,37 s correspondem a razões entre o tempo de parede e o tempo simulado de $1{,}4 \times 10^4$, $1{,}1 \times 10^4$ e $1{,}2 \times 10^4$, respectivamente, com dispersão baixa entre as dez execuções de cada

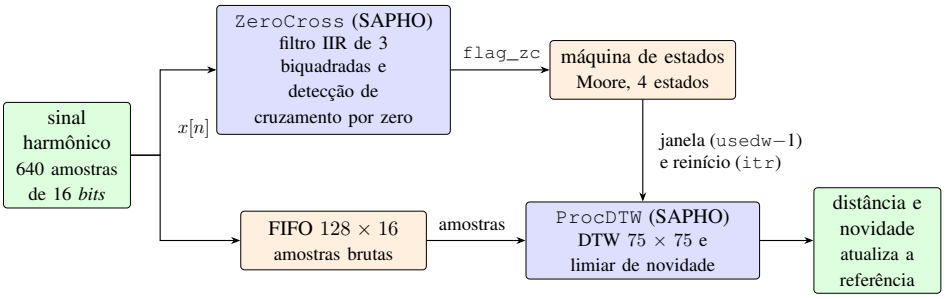


Figura 5: Estudo de caso do Modo Projeto: detector de novidade com dois processadores SAPHO. O `ZeroCross` segmenta o sinal pelos cruzamentos por zero e a máquina de estados dispara o `ProcDTW`, que drena a FIFO e mede a distância DTW até a referência.
Fonte: Autores (2026)

cenário (CV $\leq$ 4,7%). É o despejo integral das formas de onda, ao longo de dezenas de milhares de ciclos, que domina esse tempo: o mesmo `Seno` simulado em frações de segundo na Tabela 1 passa a custar dez segundos quando toda a sua atividade é registrada para análise. O custo completo é, portanto, o piso fixo da Seção 6.1 somado a um termo que cresce com o tempo simulado e domina qualquer simulação realista.

6.3 Estudo de caso: dois processadores no Modo Projeto

O `top_level` é um detector de novidade em sinais harmônicos que integra dois processadores SAPHO a uma infraestrutura convencional de *hardware*, conforme a Figura 5. O sinal de entrada alimenta em paralelo o processador `ZeroCross`, que extrai a componente fundamental com um filtro IIR de três seções biquadráticas e detecta os cruzamentos por zero, e uma fila FIFO que retém as amostras brutas. A cada cruzamento, uma máquina de estados congela a janela corrente e dispara, por interrupção, o processador `ProcDTW`, que drena a FIFO e compara a janela com uma referência pela distância de alinhamento temporal dinâmico (DTW, *Dynamic Time Warping*), técnica que alinha elasticamente duas séries temporais (Sakoe and Chiba, 1978); distâncias acima de um limiar sinalizam novidade e atualizam a referência.

O valor do cenário está no que o Modo Projeto automatiza: em um único comando, a IDE recompilou o processador ativo, reuniu as fontes Verilog do sistema, copiou e validou as memórias de programa dos dois processadores, instrumentou o banco de teste e gerou a hierarquia de módulos com o Yosys. A Tabela 2 resume a composição e a execução. Com uma amostra a cada 65,5 μs, os 7,0 ms simulados cobrem 106 das 640 amostras do sinal, o suficiente para exercitar o caminho completo de detecção com uma ativação da DTW; a extrapolação para o sinal inteiro custaria cerca de 8,7 minutos por execução, o que antecipa a limitação da Seção 7.

7. LIMITAÇÕES

Os ganhos apresentados têm fronteiras claras. A cobertura aberta descrita na Seção 5.3 alcança a modelagem, a simulação e a inspeção estrutural, mas não a implementação física: a síntese dirigida ao dispositivo, o *fitting* e a gravação dependem de informação proprietária sobre a arquitetura interna de cada FPGA e seguem a cargo do ambiente do fabricante. A avaliação restringe-se a exemplos de pequeno porte, e o custo do Icarus, interpretado, cresce com o tempo simulado nas cargas longas, motivando os simuladores compilados. Há ainda um limite inerente: a simulação valida o *modelo* do circuito, não o dispositivo físico; efeitos da implementação real, como violações de temporização pós-roteamento, não se manifestam no ambiente ideal, e a validação em ferramenta não dispensa a verificação em *hardware*. Por fim, a AURORA é especializada no ecossistema SAPHO e na cadeia YANC; a generalização para

Tabela 2: Estudo de caso do Modo Projeto: composição do sistema de dois processadores e o custo da sua execução (Dell XPS 8930, mediana de dez execuções).

Composição do sistema	
Processadores SAPHO integrados	2 (`ZeroCross`, `ProcDTW`)
Fontes Verilog reunidas	12
Instruções (`ZeroCross` + `ProcDTW`)	$141 + 274 = 415$
Palavras de dados	5831
Execução	
Tempo simulado	7,0 ms
Amostras processadas	106 de 640
Ativações da DTW	1
Tempo de parede (mediana)	87,4 s
Extrapolação (sinal completo)	$\approx 8,7$ min

Fonte: Autores (2026)

outros fluxos não é imediata.

8. CONCLUSÃO

Este trabalho apresentou a AURORA, uma IDE de código aberto que reúne, em um único ambiente, todo o fluxo de projeto do processador dinâmico SAPHO. Onde a interface anterior, a V1, exigia acionar seis ferramentas à mão para levar um algoritmo em C± da compilação até a inspeção das formas de onda, a função *Full Build* da AURORA percorre esse caminho com um único comando, e ainda acrescenta a simulação (Icarus e GTKWave) e a visualização do circuito RTL (Yosys e PRISM), ausentes na V1. Todo o percurso ocorre sobre ferramentas livres; o *software* proprietário do fabricante só volta a ser necessário na etapa final de implementação física.

A avaliação separou dois ganhos. O esforço de operação cai de seis comandos manuais para um; já o custo de máquina a AURORA não altera, pois apenas orquestra as mesmas ferramentas: fica abaixo de 0,7 s nos exemplos curtos, dominado pela elaboração do modelo, e cresce com a duração da simulação nas cargas longas. O ganho real, portanto, não é velocidade, mas a supressão dos passos manuais e a reprodutibilidade, que aproxima o pesquisador do modelo e o estudante do aprendizado, em vez da operação das ferramentas.

Como evolução, planeja-se integrar simuladores compilados (o Verilator promete reduzir em cerca de dez vezes o tempo das simulações longas), incorporar bancos de teste em Python via `cocotb` e estender o PRISM com simulação interativa.

Agradecimentos

Os autores agradecem à UFJF e ao NIPS-CERN pelo suporte e pela infraestrutura, ao CERN pela parceria científica e ao apoio da Fundação de Amparo à Pesquisa do Estado de Minas Gerais (FAPEMIG), do Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) e da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES). O presente trabalho foi realizado com apoio da CAPES, Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil, Código de Financiamento 001.

Declaração de uso de ferramentas de Inteligência Artificial generativa

Declara-se, para os fins da seção 2.8 das normas deste evento, o uso de ferramenta de inteligência artificial generativa (Claude, da Anthropic) como apoio à redação deste trabalho, especificamente na revisão do texto, no aprimoramento da clareza da escrita e na formatação. A concepção do estudo, o desenvolvimento técnico, a execução dos experimentos, a análise

dos resultados e as conclusões são de autoria integral dos autores, que revisaram e assumem responsabilidade por todo o conteúdo apresentado.

Referências

AMD (2025). Vivado Design Suite. `https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html`. Acesso em: 28 jun. 2026.

Andrade Filho, L. M., Chrysthofer Arthur, A. A., Kapisch, E. B., Castro, A. L. B. d., Rimes, S. d. M., and Libotte, G. B. (2025). Implementação de fractal em FPGA usando o processador soft-core SAPHO. In *Anais do XXVIII Encontro Nacional de Modelagem Computacional (ENMC)*.

ATLAS Collaboration (2008). The ATLAS experiment at the CERN large hadron collider. *Journal of Instrumentation*, 3:S08003.

Bybell, T. (2024). GTKWave. `https://gtkwave.sourceforge.net`. Acesso em: 28 jun. 2026.

Compton, K. and Hauck, S. (2002). Reconfigurable computing: A survey of systems and software. *ACM Computing Surveys*, 34(2):171–210.

Dirac, P. A. M. (1958). *The Principles of Quantum Mechanics*. Oxford University Press, Oxford, 4 edition.

Haykin, S. (2002). *Adaptive Filter Theory*. Prentice Hall, Upper Saddle River, 4 edition.

Intel Corporation (2025). Intel Quartus Prime Software. `https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime.html`. Acesso em: 28 jun. 2026.

Kapisch, E. B., Aguiar, M. S., Andrade Filho, L. M., and Manso Silva, L. R. (2026). SAPHO: Scalable-architecture processor for hardware optimization: An FPGA customizable implementation approach. *IEEE Access*, 14:6820–6832.

Levine, J. (2009). *flex & bison: Text Processing Tools*. O'Reilly Media.

Núcleo de Instrumentação e Processamento de Sinais (NIPS-CERN) (2026). SAPHO: Scalable-architecture processor for hardware optimization. `https://www.nipscern.com/projects/sapho`. Acesso em: 25 mar. 2026.

Ribeiro, L. d. G. (2025). Simulador de pulsos do TileCal em tempo real utilizando o processador SAPHO e FPGA. Dissertação (mestrado em engenharia elétrica), Universidade Federal de Juiz de Fora (UFJF), Juiz de Fora, MG, Brasil.

Ribeiro, L. d. G. and Andrade Filho, L. M. d. (2025). Simulador de pulsos do TileCal em tempo real utilizando o processador SAPHO e FPGA. In *Anais do XVII Simpósio Brasileiro de Automação Inteligente (SBAI)*.

Sakoe, H. and Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49.

Snyder, W. (2024). Verilator: Open-Source SystemVerilog simulator. `https://www.veripool.org/verilator`. Acesso em: 28 jun. 2026.

Williams, S. (2024). Icarus Verilog. `http://iverilog.icarus.com`. Acesso em: 28 jun. 2026.

Wolf, C. (2024). Yosys Open SYnthesis Suite. `https://yosyshq.net/yosys`. Acesso em: 28 jun. 2026.

Yang, Z., Chung, S. P., Chen, J., Zhang, R., Saltaformaggio, B., and Lee, W. (2025). CoinDef: A comprehensive code injection defense for the Electron framework. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3127–3144.

APPENDIX A: ENGLISH TITLE, ABSTRACT AND KEYWORDS

AURORA INTEGRATED DEVELOPMENT ENVIRONMENT: COMPILATION AND SIMULATION FOR THE SAPHO SOFT-CORE PROCESSOR

Abstract. *Designing FPGA-based embedded systems is essentially a computational modelling cycle: the algorithm is described, the structural model is synthesized, and the result is simulated before being committed to hardware. The SAPHO dynamic processor, which synthesizes only the resources each application requires, relied on an interface (IDE V1) that chained six tools by hand. This paper presents AURORA, a cross-platform, open-source integrated development environment built on Electron that unifies the cycle in a single operation: compilation, simulation with waveform inspection (Icarus and GTKWave) and block-level RTL visualization (Yosys and PRISM), free of proprietary software, required only for physical implementation. The evaluation shows that automation cuts operating effort from six manual commands to one, while the tool-chain machine cost, below 0.7 s, is dominated by the model-elaboration step rather than by simulation. For long simulations, evaluated up to a two-processor system, it is the simulated time that governs this cost.*

Keywords: *Digital system modelling and simulation, FPGA, Soft-core processor, Design-flow automation, AURORA IDE.*